



Neumann János Egyetem
Műszaki és Informatikai
Kar

Modell vasút sorompó vezérlő

IOT/Digitális Technika 2.

Veszelszki Tamás
PO4YGX

2026

Tartalomjegyzék:

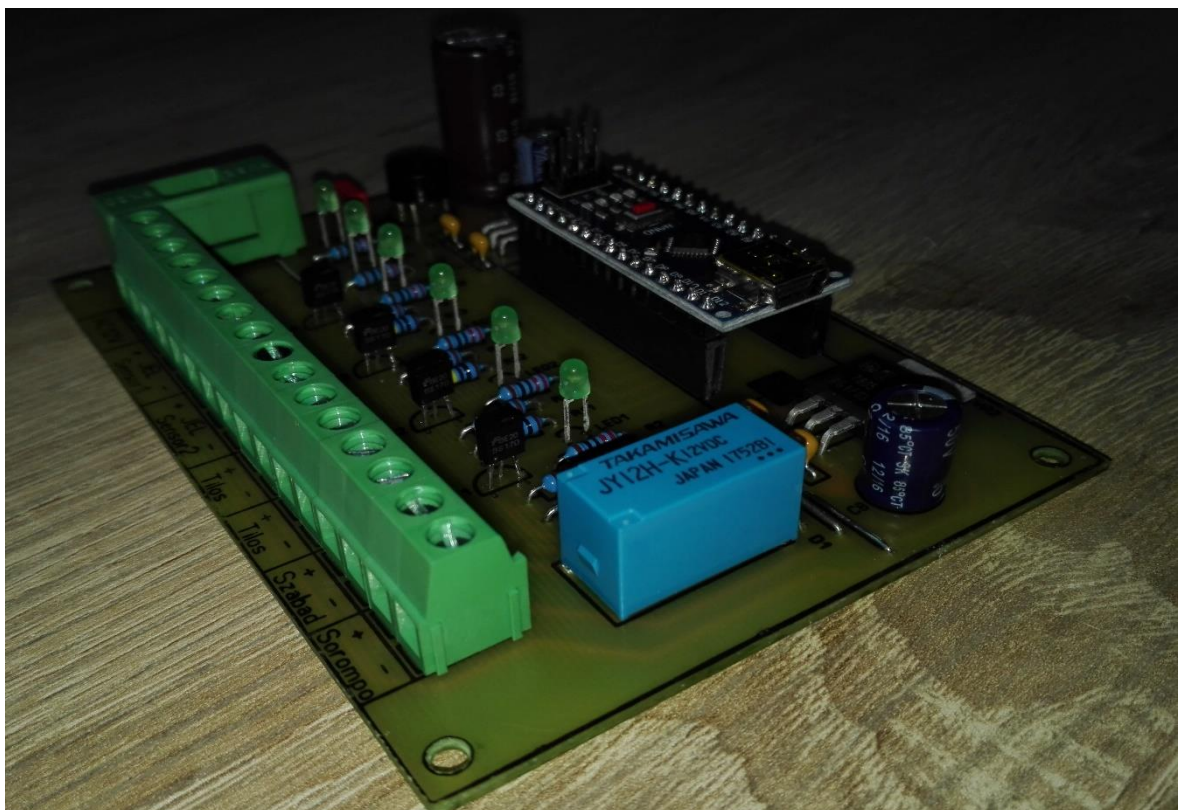
1. A PROJEKT CÉLJÁNAK ISMERTETÉSE	7
2. FELHASZNÁLT TECHNOLÓIÁK / ALKATRÉSZEK LEÍRÁSA:	10
3. A FELHASZNÁLT PROGRAM KÓD ÉS MŰKÖDÉSE:.....	11
4. TOVÁBB FEJLESZTÉSI LEHETŐSÉGEK:	15

1. A projekt céljának ismertetése

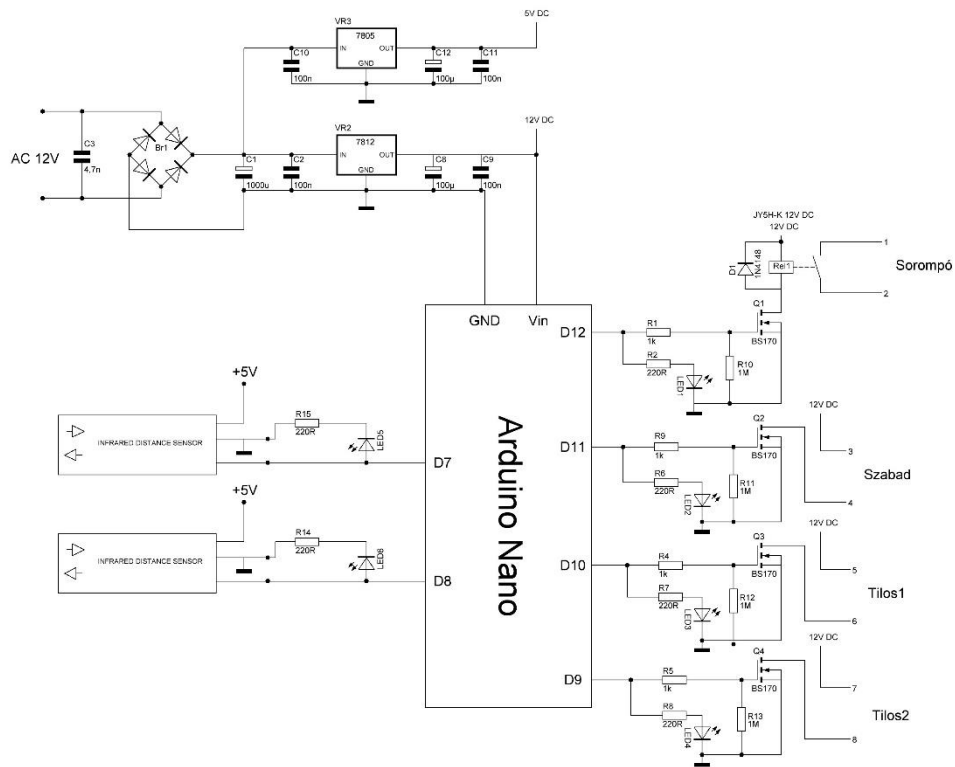
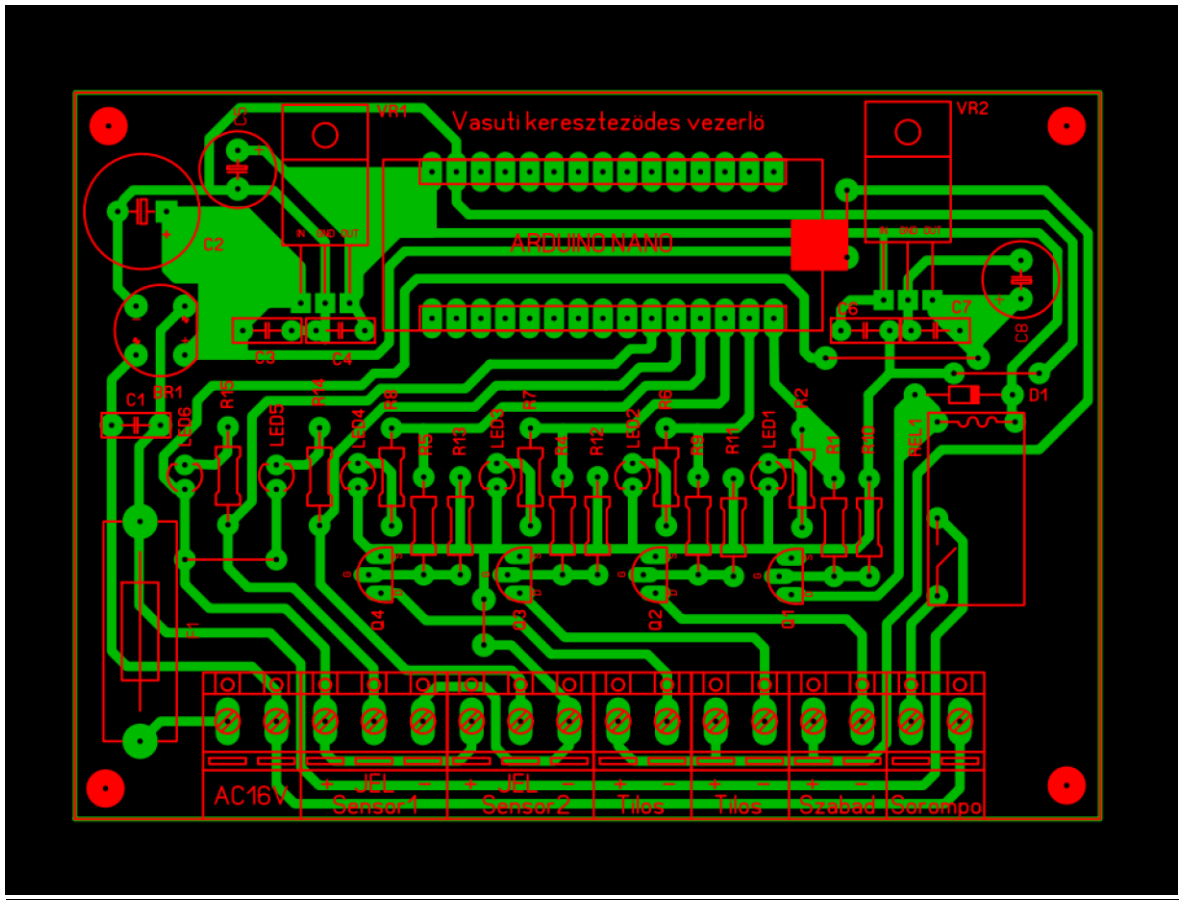
Modellvasút vasúti átjáró biztosító rendszer:

- Ez a projekt a Neumann János Egyetem üzemmérnök képzésén, az IoT alkalmazások fejlesztése / Digitális Technika II. tárgy beadandójaként készült. A projekt célja egy Arduino-alapú modell vasút sorompó vezérlő megvalósítása, amely képes 2db optikai szenzorral meghatározni a vonat közeledését és távozását az átjáróból.
- Terepasztal építése során szükség volt egy olyan céláramkörre, amely 2db optikai szenzorral képes meghatározni a vonat közeledését és távozását az átjáróból. Ezen áramkör képes vezérelni a 3 ledből álló fényjelző berendezést, valamint az elektromágnessel ellátott karos sorompót.
- Projekt GitHub link:
<https://github.com/vtomnje/Modell-vasut-sorompo-vezerlo>
- Tinkercad link:
https://www.tinkercad.com/things/dDQxzslT3md/editel?returnTo=%2Fdashboard&sharecode=qPGQrAP2DADtFy3GGFbzFw_dpVDscXudkm9PvpqtnjY
- A projekt megvalósításra került fizikai formában is, nyomtatott áramkörre, mely beépítésre is került a terepasztalba és megfelelően funkcionál.
- Mivel a kézzelfogható példány már beépítésre került egy jó barátom terepasztalába így elkészítettem Tinkercadben egy egyszerűsített változatot, hogy a működés ott is szemléltetve legyen, ehhez a linket fentebb csatoltam.

Képek az elkészült nyomtatott áramkőről:



Beültetési rajz és alkatrész jegyzékes kapcsolási rajz:

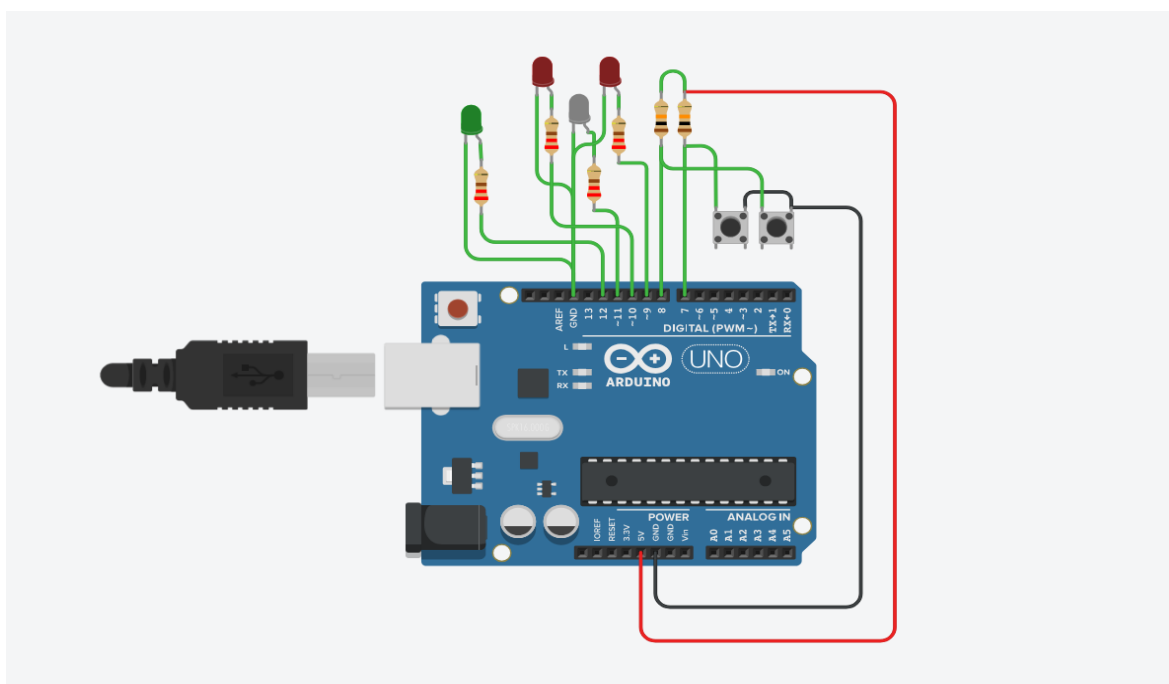


2. Felhasznált technológiák / alkatrészek leírása:

- Arduino Nano controllerrel felépített céláramkör:

- Tartalmazza a bemenetek diagnosztizálásra alkalmas ledes visszajelzést a bejövő jel függvényében, valamint a kimenetek szín és teljesítményillesztését a fogyasztókhoz. A kimenetek is diagnosztizálásra alkalmas visszajelző ledekkel vannak ellátva.
- Továbbá tartalmazza a tápegység modult, ami az AC bemeneti feszültséget egyenirányítja, pufferelemmel és szabályozza 5 illetve 12V-os szintekre.
- Ezen felül szükséges még 2db MH-B IR szenzor modul az érzékeléshez.
- A konfiguráció része még 2db elektromágneses sorompó,
- Illetve 2db 3 ledes fényjelző készülék az optikai visszacsatoláshoz.
- A további felhasznált alkatrészek (kondenzátorok és ellenállások) részletes listája értékekkel együtt megtalálható a fentebbi kapcsolási rajzon.

Tinkercad-es egyszerűsített megvalósítás képe:



3. A felhasznált program kód és működése:

//Ki- és Bemenetek konfigurálása

```
const int sens1 = 8;  
const int sens2 = 7;  
const int srmp = 12;  
const int szabad = 11;  
const int tilos1 = 10;  
const int tilos2 = 9;
```

//Villogtatáshoz szükséges változók

//LED állapotok

```
int ZledState = LOW;  
int RledState = LOW;  
int R1ledState = LOW;  
int R2ledState = LOW;
```

//Utolsó mentett idő

```
unsigned long ZpreviousMillis = 0;  
unsigned long RpreviousMillis = 0;
```

//villogási sebesség

```
const long interval = 500;
```

```
void setup() {
```

//Bemenetek beállítása

```
pinMode(sens1, INPUT);  
pinMode(sens2, INPUT);
```

//Kimenetek beállítása

```
pinMode(srmp, OUTPUT);  
pinMode(tilos1, OUTPUT);  
pinMode(tilos2, OUTPUT);  
pinMode(szabad, OUTPUT);  
}
```

```
void loop() {
```

```

//Az érzékelők állapotának vizsgálata, majd a megfelelő értéknél ugrás a megfelelő programblokkokba
if(digitalRead(sens1) == LOW){
    tilosjelzes();
}
else if(digitalRead(sens2) == LOW){
    tilosjelzes();
}
else{
    szabadjelzes();
}
}

void szabadjelzes() {
    //Szabadjelzésnél használt programblokk, a sorompó nyitva, csak a fehér LED villog
    digitalWrite(srmp, LOW);
    digitalWrite(tilos1, LOW);
    digitalWrite(tilos2, LOW);

    unsigned long ZcurrentMillis = millis();

    if (ZcurrentMillis - ZpreviousMillis >= interval) {
        // az utolsó ledvillogás mentése
        ZpreviousMillis = ZcurrentMillis;

        // Ha LED kiakcsolva, akkor bekapcsol és fordítva
        if (ZledState == LOW) {
            ZledState = HIGH;
        } else {
            ZledState = LOW;
        }

        // LED Be- és Kikapcsolása ZledState alapján
        digitalWrite(szabad, ZledState);
    }
}

void tilosjelzes() {
    // TILOS jelzésnél használt programblokk, ilyenkor a sorompó lezár, a piros LED-ek felváltva
    villognak.
    digitalWrite(srmp, HIGH);

```

```

digitalWrite(szabad, LOW);

unsigned long RcurrentMillis = millis();

if (RcurrentMillis - RpreviousMillis >= interval) {
    // az utolsó ledvillogás mentése
    RpreviousMillis = RcurrentMillis;

    // Ha LED kiakcsolva, akkor bekapcsol és fordítva
    if (RledState == LOW) {
        RledState = HIGH;
    } else {
        RledState = LOW;
    }

    // LED Be- és Kikapcsolása ZledState alapján
    digitalWrite(tilos1, RledState);
    digitalWrite(tilos2, !RledState);
}
}

```

A kód működése:

A beépített 2 db szenzor jele a void loop() függvényben kerül kiértékelésre:

if – else if – else függvényrendszerben kvázi vagy kapcsolatban van a 2 szenzor, tehát mindegy, hogy melyik irányból jön a vonat, a program bármelyik szenzor jelétől beugrik a tilosjelzes() függvénybe.

A tilosjelzes() függvény villogtatja felváltva a fényjelző készülék piros ledjeit felváltva, valamint lecsukva (meghúzva) tartja a sorompókat.

Ez az állapot addig marad így, ameddig a 2 szenzor előtt a szerelvény eltávolodik.

Ekkor az else ágban a szabadjelzes() függvény aktiválódik, amelynek a szerepe a fényjelző készülék fehér ledjének a villogtatása, valamint a sorompó felengedése.

Az időzítéseket nem a `delay()` függvényel valósítottam meg, hanem ciklusidő barátabb megoldást választottam, ugyanis a `delay()` függvény feltartja a processzort, ameddig várakozik, így addig nem tud a processzor más feladattal foglalkozni.

Erre a megoldás, hogy lekértem a processzor aktuális idejét a `millis()` utasítással, ezután a jelenlegi idő és az utolsó idő különbségét összehasonlítottam a késleltetni kíván idővel, amikor ez a feltétel teljesült, akkor csináltam meg az adott ponton a feladatot.

4. Tovább fejlesztési lehetőségek:

A már meglévő 2 db szenzor prelegés mentesítése:

Ritkán, de előfordulhat, hogy a beszerelt pozícióban a szerelvényen átláthat a szenzor (pl speciális, emelt szerelvény, nagy ablakok, stb), ennek kiküszöbölésére érdemes lenne a kódba beleépíteni szenzoronként egy késleltetést, ami akkor aktiválnódna, ha a szenzor kimenete 0.